THE NATIONAL UNIVERSITY OF LESOTHO

Department of Mathematics and Computer Science
Faculty of Science and Technology

CS4433: SOFTWARE ENGINEERING

PHASES 5–8

Verification and Validation • Configuration Management and QA
Deployment • Maintenance

MODERATED LEARNING MANAGEMENT SYSTEM

Prepared by
Bakoena Thamae
202321319
BSc Information Systems

National University of Lesotho

1 November 2026

Document Status and Submission Notes

This is the Phase 5–8 technical report supplement for the Moderated LMS individual assignment. It consolidates the verification, quality assurance, deployment and maintenance artefacts prepared for the C++17 offline application. It is intended to be read with the Phase 1 project management plan, Phase 2 requirements specification and Phase 3 design document.

Evidence status: Local build, database-initialisation, schema-parity, migration and business-rule smoke checks are recorded as completed in the development container. The Catch2 suite, GitHub Actions run, Windows target-machine acceptance tests, 800-student performance test, usability observations, backup/restore timing and release tags remain pending until actually executed and recorded by the student.

Repository: https://github.com/BakoenaT/Moderated-Learning-Management-System

Implementation language: C++17. Runtime persistence: SQLite. Password hashing: OpenSSL PBKDF2-HMAC-SHA256. Build system: CMake.

Abstract

This report presents the verification and validation approach, configuration management and quality assurance arrangements, deployment documentation, and maintenance response for the Moderated Learning Management System (LMS) designed for a secondary school. The application supports four roles—Administrator, Lecturer, Head of Department (HoD) and Student—and operates offline using a local SQLite database. Phase 5 applies the V-Model to the system requirements and design, defines equivalence partitions and boundary tests, and records available automated and manual evidence. Phase 6 documents the GitHub Flow branching approach, continuous-integration workflow, code metrics, static-analysis evidence and self-review actions. Phase 7 defines the offline deployment architecture, technician installation procedure, role-based user guidance and v1.0 release notes. Phase 8 analyses and implements Change Request CR1, which enables a lecturer to grant an extension to a whole class and makes the effective deadline visible to students. The report distinguishes measured results from pending activities so that no unexecuted test, CI run or release is represented as completed.

Contents

- 1. Introduction and implementation baseline
- 2. Phase 5 — Verification and Validation
- 3. Phase 6 — Configuration Management and Quality Assurance
- 4. Phase 7 — Deployment
- 5. Phase 8 — Maintenance (CR1)
- 6. Cross-phase requirements traceability
- 7. Professional, legal and ethical considerations
- 8. Conclusion and outstanding actions
- References
- Appendix A — System and acceptance test cases and actual outcomes
- Appendix B — Defect log and static-analysis results
- Appendix C — CR1 regression tests and actual outcomes
- Appendix D — Repository artefact inventory and uploaded evidence
- Appendix E — Executed test results and coverage record
- Appendix F — Phase 6–8 verification and release readiness

In Microsoft Word, update the table of contents if page numbers or section positions change after editing.

1. Introduction and Implementation Baseline

The Moderated LMS is an offline C++17 console application intended for use on a designated school workstation. It manages departments, courses, user accounts, lecturer-course assignments, student enrolments, assignments, submissions, marking, moderation, release of results and audit information. The four application roles are Administrator, Lecturer, Head of Department and Student.

The approved Phase 2 requirements are the principal verification basis. They include FR-01 to FR-18, NFR-01 to NFR-09 and BR-01 to BR-08. Important constraints include local/offline operation, role- and department-scoped access, password hashes rather than plaintext passwords, mark auditability, late penalties, individual extensions, resubmission controls, and the requirement that students see marks only after HoD moderation and explicit release.

The Phase 4 implementation uses SQLite at runtime, accessed through the Db data-access component and parameterised statements. The Phase 3 MySQL schema remains a design artefact; it is not the runtime database for this assignment. Password hashes use OpenSSL PBKDF2-HMAC-SHA256 with a random salt. Marking, moderation, release and audit operations use database transactions where required.

The Phase 5–8 package refactors date and extension rules into src/business_rules.cpp/.hpp to make them independently testable. The application code and schema in the package represent the updated implementation baseline prepared for these later phases. The student should ensure that the updated files are pushed to the GitHub repository before relying on the GitHub Actions workflow or release tags.

2. Phase 5 — Verification and Validation (12 Marks)

2.1 Purpose and test basis

The purpose of verification and validation is to establish whether the implemented LMS conforms to its specified requirements and whether the delivered workflows meet stakeholder needs. Verification checks the implementation against the SRS, architecture, detailed design, database schema and use-case specifications. Validation evaluates the complete system with representative users and realistic school scenarios.

The strategy follows the V-Model. Each development artefact is paired with a corresponding test level. Tests are evidence-producing activities: a case is marked as passed only after it has actually been executed, the actual result has been recorded, and suitable evidence has been retained.

2.2 V-Model mapping, entry and exit criteria

Level	Development artefact verified	Test focus	Entry criteria	Exit criteria
Unit	Detailed class specifications, late-penalty strategy, submission/extension rules and validation routines	Date parsing; late-day boundaries; adjusted maximum; resubmission deadline; class extension policy	Business-rule module compiles; test data and expected results reviewed	Planned unit cases executed; critical defects fixed or accepted; summary and coverage saved
Integration	Architecture, class-to-table mapping, data-access layer and SQLite schema	Prepared statements; foreign keys and unique constraints; transaction boundaries; class-extension writes; audit persistence; file metadata	Unit tests pass; clean test database available; migrations applied	Interfaces and database cases executed; no partial write after failed transaction; referential integrity passes
System	Use-case model and behavioural diagrams for login, assignments, submissions, marking, moderation and release	End-to-end role workflows; exception paths; offline operation; access boundaries; recoverability	Integrated executable built; role accounts and representative records seeded	Every Must use case has evidence; exception paths tested; high/critical defects closed; report complete

Level	Development artefact verified	Test focus	Entry criteria	Exit criteria
Acceptance	Fully dressed use cases and stakeholder needs	Principal/administrator, lecturer, HoD, student and IT officer scenarios; usability and performance	System tests passed; manuals available; representative users and target machine prepared	Acceptance criteria evaluated; deviations and sign-off recorded; no release-blocking defect remains

2.3 Test design techniques

Equivalence partitioning groups inputs expected to behave in the same way. The test design includes the following classes:

- Authentication: valid active account; wrong password; unknown username; inactive account.
- Role and scope: permitted role/action; authenticated but wrong role; correct role but wrong course, department or student scope.
- Marks: below zero; valid inclusive range from zero to the adjusted maximum; above the adjusted maximum; non-numeric input.
- Submission files: existing regular file; missing path; directory instead of a file; inaccessible file. The current console UI does not yet enforce a file-extension and size allow-list.
- Extensions: assigned lecturer and enrolled class; unassigned lecturer; assignment owned by another lecturer; empty class; invalid deadline; empty reason; valid revised deadline.

Boundary-value analysis focuses on the late penalty. The SRS assumption is calendar days. For an assignment maximum of 100 marks, the maximum permitted mark is 100 on the deadline date, 90 after one late calendar day, 80 after two days, 70 after three days, and zero after more than three days.

Late days	Expected maximum mark (out of 100)	Interpretation
Earlier than deadline / -1	100	Not late; negative lateness is treated as zero
0	100	Submitted on deadline date
1	90	First late day
2	80	Second late day
3	70	Final penalised day
4	0	Beyond the three-day penalty window

Resubmission boundary: a first attempt can be accepted late under the late-submission policy. Once a student has an existing attempt, a resubmission must be made on or before the effective deadline. The latest permitted attempt becomes active while earlier attempts remain in the database as history.

2.4 Automated unit and integration tests

The Catch2 v3 test suite is located in tests/test_business_rules.cpp and covers the business-rule and CR1 scenarios. Its source is present, but it was not executed in this local environment: CMake FetchContent could not resolve github.com to download Catch2. The separately compiled manual assertion harness was executed and passed; it does not replace the Catch2/CTest and GitHub Actions evidence required for final submission.

The project also includes tests/manual_business_rules_smoke.cpp, a standalone assertion harness used to produce local evidence. This harness supplements, but does not replace, the required Catch2 suite and CI coverage report.

2.5 Local build, smoke test and coverage results

The following activities were executed on 1 November 2026 in the development container (Linux x86-64), not on the student's Windows target workstation. In addition to build, database and rule-level checks, a scripted console end-to-end CR1 scenario was run across an application restart.

Activity	Environment / tool	Result	Evidence and limitation
CMake configure (production build; testing disabled)	CMake 3.31.6; GCC 14.2.0; SQLite 3.46.1; OpenSSL found by CMake	PASS	Fresh production configure completed; BUILD_TESTING=OFF because Catch2 could not be fetched in this environment.
Application compilation	GCC 14.2.0; C++17; Linux x86-64	PASS	Clean build compiled and linked moderated_lms and lms_business_rules; no warning in captured output.
Database initialisation	Built moderated_lms --init-only	PASS	Ten application tables initialized; extension_scope present.
SQLite schema script	SQLite 3.46.1 / Python sqlite3	PASS	Ten tables; runtime and standalone schema columns/FKs identical; no foreign-key violations.
Runtime/schema parity	Python sqlite3	PASS	Ten runtime table/column definitions match schema; foreign_key_check had no violations.
Legacy database migration	SQLite 3.46.1; legacy extension table fixture	PASS	Added extension_scope as INDIVIDUAL and preserved the existing extension row.
Business-rule smoke assertions	GCC 14.2.0; SQLite; gcov	PASS	PASS: business rule smoke assertions; includes later class extension, no-shortening and rollback.
Business-logic coverage	gcov 14.2	84.18% lines; 78.91% branches	84.18% of 158 executable lines; 78.91% of 294 branches executed at least once. Preliminary smoke-harness result only.
Clang Static Analyzer	Clang 17	PASS	No diagnostics from business_rules.cpp and main.cpp. clang-tidy has not been executed.
Catch2 suite / GitHub Actions	Catch2 v3.7.1 FetchContent; CMake	BLOCKED / PENDING	Catch2 clone failed because github.com could not be resolved; workflow and tests need a network-enabled CI run.
Target workstation system/acceptance, 800-student performance, usability, backup/restore	Student Windows/school PC	NOT RUN	Record actual outcomes in the acceptance test and evidence sheets.
CR1 console end-to-end and restart	C++17 app; two process runs; persistent SQLite; Linux x86-64	PASS	Admin provisioned course/users; lecturer granted class + individual extensions; after restart each student saw correct effective deadline and label. See CR1_END_TO_END_TEST_OUTPUT.txt.

Coverage qualification: 84.18% line coverage and 78.91% branch coverage are measured for the business-logic translation unit using the independent local smoke harness. They are not the final Catch2/CTest CI result. The assignment's formal 70% line-coverage objective remains subject to a successful CI run and retained coverage artifact.

2.6 System and acceptance testing

The system and acceptance test set is derived from the use cases and SRS. Actual outcomes are recorded for valid login, core account/course/assignment setup, individual extension, and the CR1 class-extension display workflow. All other cases remain Not Run or Partial where only the business-rule layer was exercised. The full matrix appears in Appendix A and SYSTEM_ACCEPTANCE_TESTS.csv.

2.7 Defect management

Defects are classified as Critical, High, Medium or Low. Critical defects compromise confidentiality or integrity or prevent startup. High defects break a Must requirement or role security. Medium defects affect a Should requirement or important workflow. Low defects are cosmetic or usability issues. A defect is closed only after a repeatable retest passes.

ID	Defect / impact	Fix implemented	Local retest / remaining status
D-001	Resubmission after the effective deadline was not explicitly rejected.	Added a strict resubmission-time predicate; first submissions retain the late-work policy.	Manual boundary assertions passed: deadline accepted; one second after rejected. Catch2/CI pending.
D-002	Invalid calendar/date-time input could be treated as zero late days.	Added strict Gregorian date and time validation in the business-rule module.	Manual parser assertions rejected invalid leap day, month and hour. Catch2/CI pending.
D-003	No whole-class extension or visible extension source in student assignment list.	Added class-wide grant, scope field, effective deadline display, and migration.	CR1 SQLite and console E2E passed for class grant, individual exception, restart display and rollback. Catch2/CI pending.

The initial defect register contains three observations: a missing explicit resubmission boundary, invalid date input being treated as zero late days, and the absence of class-wide extensions and extension-source visibility. Each fix passed the corresponding local manual/business-rule retest; CI and target-platform retesting are still pending, so the records are not treated as release-final.

3. Phase 6 — Configuration Management and Quality Assurance (6 Marks)

3.1 Repository and baseline

The source repository is https://github.com/BakoenaT/Moderated-Learning-Management-System and its default branch is main. The main branch is intended to contain the reviewed, buildable baseline. The exact commit SHA and release tags must be copied from GitHub at the date of submission; they are not hard-coded in this report because they can change after new commits.

3.2 Branching and commit strategy

- Use GitHub Flow with main as the protected/reviewed baseline. Avoid direct feature work on main.
- Create one focused branch for each coherent change, for example feature/phase5-tests, feature/phase6-ci, feature/phase7-deployment-docs and feature/cr1-class-extension.
- Commit small, meaningful units with imperative messages, such as “Add deadline boundary unit tests” or “Implement class-wide extension grant”.
- Open a pull request to main. Review code, tests and traceability; merge only when the required CI checks pass.
- Create v1.0 and v1.1 tags only from reviewed and accepted commits. Do not create a release tag merely because release notes have been drafted.

3.3 Example Git workflow

```
git checkout main
git pull origin main
git checkout -b feature/cr1-class-extension
# implement and test the change
git add .
git commit -m "Implement class-wide extension grant"
git push -u origin feature/cr1-class-extension
# open a pull request to main and merge after review and CI pass

git checkout main
```

```
git pull origin main
git tag -a v1.1 -m "Moderated LMS v1.1 - class-wide extensions"
git push origin v1.1
```

3.4 Continuous integration

The workflow is defined in .github/workflows/ci.yml. It triggers on pushes to main, feature/** and maintenance/** branches and on pull requests targeting main. The build-test-coverage job checks out the repository, installs CMake, GCC, SQLite, OpenSSL and gcovr, configures a C++17 build with tests and coverage enabled, builds, runs CTest, generates a gcovr XML report for src/business_rules.cpp, and enforces the 70% line-coverage threshold. Test logs and coverage are uploaded as workflow artefacts. A separate static-analysis job installs clang-tidy and analyses src/business_rules.cpp.

CI status: The workflow definition is prepared in the package. The first GitHub Actions run must occur after the current phase files are pushed to GitHub. This report does not claim a successful GitHub Actions run. Retain the workflow URL, commit SHA, job status, test count and downloaded coverage artefact as submission evidence.

3.5 Static analysis, code metrics and coding standard

The C++ build enables /W4 for MSVC and -Wall -Wextra -Wpedantic for GCC/Clang. The .clang-tidy configuration is included, and CI is configured to run clang-tidy on the independently testable business-rule module. The coding standard uses descriptive lowerCamelCase function and variable names, PascalCase types, one responsibility per function/class, parameterised SQL, checked return codes, RAII where appropriate, transaction handling for multi-row changes, validation before persistence, and no plaintext passwords.

Metric / function	Value	Method / qualification
src/main.cpp	171 physical lines	wc -l; compact/minified formatting means physical lines understate logical statements.
src/business_rules.cpp	228 physical lines	wc -l; business logic isolated for independent verification.
src/business_rules.hpp	32 physical lines	wc -l.
tests/test_business_rules.cpp	173 physical lines	wc -l; Catch2 suite source; test execution is pending due FetchContent network restriction.
tests/manual_business_rules_smoke.cpp	39 physical lines	wc -l; standalone assertion harness executed locally.
Total	643 physical lines	Current Phase 5–8 source/test snapshot; sum of the five listed files.
calendarDaysLate	1 (estimated)	Manual structural estimate; not a Lizard result.
lateAdjustedMaximum	4 (estimated)	Manual structural estimate; not a Lizard result.
submissionTimeAllowed	2 (estimated)	Manual structural estimate; not a Lizard result.
grantWholeClassExtension	13 (estimated)	Manual structural estimate; validate with Lizard if required.

Static-analysis and compiler evidence: the GCC 14.2.0 production build completed with configured warning flags and no warning in captured output. Clang 17 Static Analyzer completed for src/business_rules.cpp and src/main.cpp with no diagnostics. clang-tidy and Lizard were not installed locally and remain pending. CMake FetchContent for Catch2 was blocked because this environment could not resolve github.com. Cyclomatic complexity values in this report are manual structural estimates, not tool-generated scores.

3.6 Self-code review

The self-review covers requirement traceability, role enforcement, input validation, parameterised queries, transaction handling, exception handling, password protection, offline operation, SQLite schema/code consistency, coverage and documentation.

Review area	Finding / action recorded
Requirement coverage	Added tests for late-day boundaries, resubmission deadline and CR1 class extension.
Security and role scope	Class-extension operation validates assignment ownership before writing; student and HoD scopes remain checked by role workflows.
Input validation	Business rules reject malformed date/time values, invalid mark limits and missing extension reasons.
Data integrity	Class extension uses a single SQLite transaction; existing individual extension rows remain intact.
Persistence	Added extension_scope to clean schema and additive migration for existing DB files.
Error handling	Invalid assignment/lecturer and empty class are rejected and handled by menu-level exception handlers.
Maintainability	Date, penalty and extension rules separated into a unit-testable translation unit.
Documentation	README, test plan, user/installation guides, CR1 analysis and v1.1 release notes updated.

The student must complete reviewer name/initials and final commit SHA after reviewing the exact GitHub commit. CI run URL, clang-tidy/Lizard outputs and target acceptance records remain outstanding.

4. Phase 7 — Deployment (4 Marks)

4.1 Deployment model

The deployment target is a designated school workstation running Windows or Linux. The application is a local C++17 console executable. SQLite stores application records in moderated_lms.db, while uploaded student files are stored beneath a controlled submissions/ directory. OpenSSL provides the password hashing functionality. Backups copy the database and submission directory together to encrypted, access-controlled removable media. The system is designed for offline operation and does not provide a browser/mobile client or remote multi-user service.

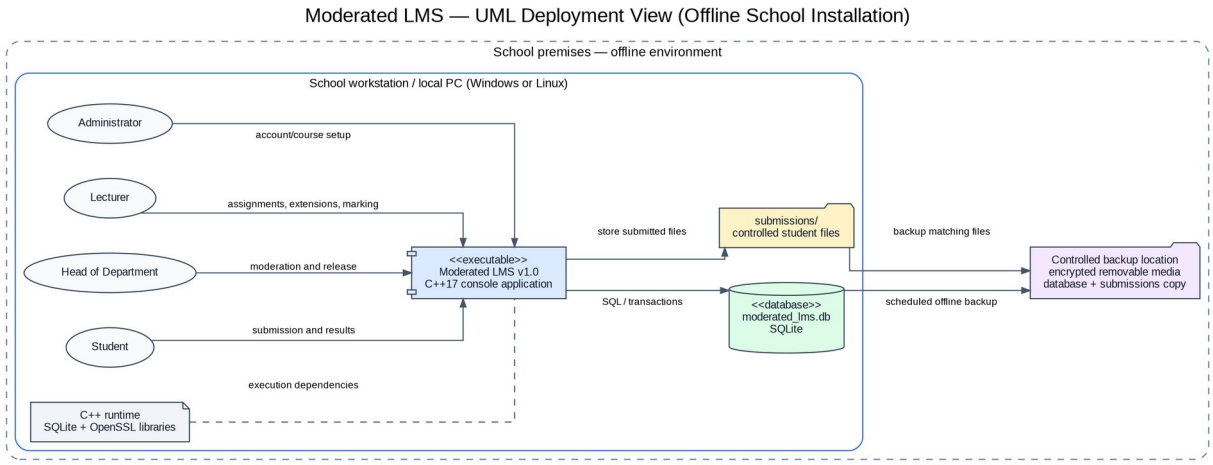


Figure 1. UML deployment view for the offline school workstation installation.

4.2 Technician installation guide

Scope and prerequisites

The deployment guide assumes one designated workstation is the authoritative host. Do not put the SQLite database on a general network share or permit multiple independent database copies to be edited. The system requires Windows 10/11 64-bit or a supported Linux desktop, a C++17 runtime/build environment, CMake 3.20 or later, SQLite development libraries and OpenSSL development libraries. Catch2 is needed for developer/CI tests but not for the production build.

Windows build procedure

1. Install Visual Studio Build Tools with Desktop development with C++, MSVC x64/x86 tools and a Windows SDK.
2. Install CMake 3.20 or later.
3. Install vcpkg and required libraries, for example: `vcpkg install sqlite3:x64-windows openssl:x64-windows`.
4. Open Developer PowerShell for Visual Studio and change directory to the repository root.
5. Configure and build the production executable using the commands below.
6. Copy the executable and required OpenSSL runtime DLLs (if dynamically linked) into the approved application folder. Run it from a writable data folder with restricted permissions.

```
cmake -S . -B build -G "Visual Studio 17 2022" -A x64 `
  -DCMAKE_TOOLCHAIN_FILE="C:/vcpkg/scripts/buildsystems/vcpkg.cmake" `
  -DBUILD_TESTING=OFF
cmake --build build --config Release
```

Ubuntu/Debian build procedure

```
sudo apt update
sudo apt install -y build-essential cmake libsqlite3-dev libssl-dev
cmake -S . -B build -DCMAKE_BUILD_TYPE=Release -DBUILD_TESTING=OFF
cmake --build build --parallel
./build/moderated_lms --init-only
```

First-run setup and data security

7. Create a dedicated operating-system account for the LMS and restrict application-folder permissions to authorised staff.
8. Run the application from the approved data directory. It creates moderated_lms.db and the submissions/ folder as needed.
9. The current development bootstrap account is username admin and initial password ChangeMe123!. Do not use the system with real learner data while this shared default credential remains active. The current console application does not yet provide a password-change screen; secure credential provisioning is required before live use.
10. Use only dummy accounts for demonstration until the default credential risk is resolved. Verify each account role and department assignment.
11. Ordinary students must not browse the raw SQLite database or submission directory. Never email the database or store it in a public cloud folder.

Backup and restore

12. Ask users to log out and close the LMS cleanly.

13. Copy moderated_lms.db and the complete submissions/ directory together to encrypted, access-controlled removable media. Keep a dated backup log.
14. Do not back up only the database because records contain file paths to submitted files.
15. To restore, close the application, move the damaged database/submissions directory aside, copy both items from the same backup set and restore folder permissions.
16. Start the app and verify a known course, assignment, submission record, file, mark and audit entry. Record elapsed time; the Phase 2 target is restoration within 15 minutes.
17. Test restoration periodically on a separate copy, never by overwriting the live database.

Acceptance and handover checklist

- ☐ Clean build on the target workstation succeeds.
- ☐ Test logins for all four roles work.
- ☐ Role and department restrictions are demonstrated.
- ☐ Core workflows operate without internet access.
- ☐ Database and submissions folder permissions are checked.
- ☐ Backup and restore are completed and timed.
- ☐ Default password provisioning risk is resolved before real learner data is entered.
- ☐ Technician receives repository link, build steps, backup instructions and limitations.

4.3 User manual — Version 1.0

Getting started (all users)

1. Start the `moderated_lms` application from the designated school workstation.
2. Choose `1 Login` at the opening menu.
3. Enter the username and password supplied by the administrator. Password input is visible while typed; use the application only in a private controlled environment.
4. Choose the menu item for your role. Enter `0` to log out of the role menu and `0` again to exit.
5. Do not access the SQLite database or submissions folder directly.

Administrator

1. Choose `1 List users` to inspect registered accounts.
2. Choose `2 Create user`; select Administrator, Lecturer, HoD or Student; enter username, full name, optional email, initial password (at least 10 characters) and department ID where appropriate.
3. Choose `3 Activate/deactivate user` to disable or re-enable an account. Confirm the user ID carefully.
4. Choose `4 Create department`; record the department ID.
5. Choose `5 Create course`; enter department ID, course code, title and description. Record the course ID.
6. Choose `6 Assign lecturer`; enter Lecturer user ID and course ID.
7. Choose `7 Enrol student`; enter Student user ID and course ID.
8. Choose `8 List courses` to verify academic structure.

Administrator safeguards: use unique accounts; select the correct role and department; deactivate leavers promptly; do not share administrator credentials.

Lecturer

1. Choose `1 My courses` to view courses assigned to your account.

2. Choose `2 Create assignment`; select a course ID and enter title, instructions, deadline (`YYYY-MM-DD HH:MM:SS`) and a positive maximum mark.
3. Choose `3 Grant individual extension`; enter assignment ID, enrolled student ID, revised deadline and reason. Only the lecturer assigned to the assignment may grant it.
4. Choose `4 View submissions` to inspect submission IDs, student IDs, versions and status.
5. Choose `5 Mark submission`; enter a submission ID, mark within the late-adjusted maximum, written feedback and reason. The mark remains hidden until HoD moderation and release.

Version 1.0 supports individual extensions. Whole-class extensions are introduced by maintenance release v1.1.

Head of Department

1. Choose `1 Pending marks in my department` to inspect marks awaiting moderation or returned for correction.
2. Choose `2 Approve/return mark`; enter the mark ID, choose A to approve or R to return, and provide a moderation reason.
3. Choose `3 Release approved mark` and enter an approved mark ID. Only approved marks in your department can be released.
4. Ensure moderation and release decisions are justified. Students must not see a mark before explicit release.

Student

1. Choose `1 Courses/assignments` to view only enrolled courses and their assignments/deadlines.
2. Choose `2 Submit or resubmit file`; enter assignment ID and the full file path on the school workstation. Keep the original file until confirmation.
3. A first attempt may be accepted late under the configured policy. Resubmissions are permitted only on or before the effective deadline. Earlier attempts remain in history.
4. Choose `3 Released results` to view only your own released marks and written feedback.

Support and safe use

Contact the administrator/IT officer if login fails, a file cannot be submitted, or a course/assignment is missing. Do not repair data by editing the database. Report suspected incorrect marks or unauthorised access. The v1.0 console does not provide password self-service or an accessibility-tested graphical interface; assisted access should be arranged where necessary.

4.4 Release notes — v1.0

Release type: initial baseline deployment. Create the v1.0 tag only on the accepted commit after successful build, test and review.

- Offline C++17 console application with Administrator, Lecturer, HoD and Student roles.
- Local SQLite persistence through the Db data-access component.
- Salted PBKDF2-HMAC-SHA256 password hashes using OpenSSL.
- Department/course management, lecturer assignment and student enrolment.
- Assignment creation, individual extensions, file submission and resubmission history.
- Late penalty, lecturer marking/feedback, HoD moderation and explicit result release.
- Audit records for mark creation/changes, moderation and release.

Known v1.0 limitations: no browser/mobile interface, online payment, video conferencing, automated grading or plagiarism detection (out of scope); no password-change/reset menu; console

password entry is visible; upload extension/size allow-list is not enforced; backup/restore requires a timed exercise; deployment is intended for one designated local workstation, not a shared network filesystem. Before replacing the executable, back up the database and submissions folder together and verify sample records after upgrade.

5. Phase 8 — Maintenance: Change Request CR1 (6 Marks)

5.1 Change request and classification

> CR1: “A lecturer should be able to extend the deadline for a whole class, not just an individual student, and extensions should be shown on the student’s assignment list.”

CR1 is classified as perfective (enhancement) maintenance. It introduces a capability that did not exist in the v1.0 individual-extension workflow and improves the visibility of revised deadlines to students. It is not primarily corrective maintenance because individual extensions were already part of the agreed feature set.

5.2 Policy decision for overlapping extensions

The assignment brief does not specify how a whole-class extension interacts with an existing individual extension. The implemented rule is that a class grant is recorded for each currently enrolled student who does not have an individual extension, or whose existing extension is already class-scoped. A pre-existing individual extension is preserved as a student-specific exception. The lecturer may subsequently replace that student’s individual extension using the individual-extension operation. This prevents a general class grant from unintentionally shortening or removing a separately approved personal arrangement.

A class extension applies to students enrolled in the course at the time the lecturer grants it. Students enrolled later do not automatically inherit a historical class grant; the lecturer can issue a new class grant if needed. These two policy decisions require supervisor/stakeholder confirmation before final acceptance.

5.3 Impact analysis

Artefact	Before CR1	Required change
SRS FR-08	Lecturer grants an individual student extension.	Extend to individual and whole-class operations; show revised/effective deadline on student assignment list.
BR-02 / lecturer scope	Lecturer acts only on assigned courses.	Retain; class grant must validate assignment ownership.
BR-06 / deadline	Individual extension replaces original deadline for one student.	Class grant sets effective deadline for enrolled students without individual override; individual remains exception.
Use case model	Grant individual extension.	Add Grant whole-class extension and show effective deadline in student assignment view.
Activity / sequence models	One student receives extension.	Add roster loop, individual exception logic, transaction commit and rollback paths.
Class design	Individual extension workflow.	Add grantWholeClassExtension business rule and retain independently testable policy functions.
SQLite schema	One row per assignment/student.	Add extension_scope (`INDIVIDUAL` or `CLASS`), keep unique assignment/student key, migrate old databases.
Student assignment query	Original assignment deadline displayed.	Display effective deadline and Class extension / Individual extension / No extension label.
Source code	Individual extension menu only.	Add lecturer class-extension action, transaction operation and effective-deadline display.
Tests	Individual extension only.	Test roster coverage, authorisation, individual overrides, empty class, atomic rollback and display.
Deployment/manual	v1.0 guidance only.	Add v1.1 lecturer and student instructions and release notes.

5.4 Data design and transaction behaviour

CR1 reuses the existing extension_grant table rather than introducing a second, competing deadline table. It adds extension_scope with allowed values INDIVIDUAL and CLASS. The existing unique key (assignment_id, student_id) remains. The class-extension operation validates the assignment owner and revised deadline, reads the current enrolment roster, and applies all eligible records inside one BEGIN IMMEDIATE transaction. If a database write fails, the operation rolls back the whole class update.

For each enrolled student, the effective deadline is extension_grant.deadline when a grant exists; otherwise assignment.deadline applies. The student assignment list displays the effective date and extension type. Granting an extension does not itself change marks, submissions, moderation status or result release.

5.5 Database migration and backward compatibility

For a clean database, database/schema.sql creates extension_scope with default INDIVIDUAL. At startup, the Db constructor checks PRAGMA table_info(extension_grant). If an existing Phase 4 database does not contain the column, an additive ALTER TABLE migration adds it with default INDIVIDUAL. Existing individual rows are therefore preserved. Back up moderated_lms.db and submissions/ before applying v1.1.

5.6 Updated design responsibilities

Component	Updated responsibility
App::lecturerMenu	Presents the whole-class action and passes validated input to the business-rule operation.
lms::grantWholeClassExtension	Validates assignment ownership and deadline, reads enrolled students, applies eligible updates in one SQLite transaction and returns affected-row count.
lms::calendarDaysLate	Provides strict calendar-day calculation shared by application and tests.
lms::lateAdjustedMaximum	Applies the 10% per late calendar day rule, capped at three days.
lms::submissionTimeAllowed	Enforces the effective-deadline rule for resubmissions.
extension_grant.extension_scope	Distinguishes INDIVIDUAL and CLASS grants.
Student assignment-list query	Returns effective_deadline and extension_type for display.

Updated UML source files are supplied separately: docs/phase8/cr1_use_case_update.puml, cr1_activity.puml and cr1_sequence.puml. They respectively represent the use-case update, class roster/transaction activity and lecturer-to-database sequence.

5.7 Lecturer and student manual addendum — v1.1

Lecturer: grant a whole-class extension

1. Sign in as the Lecturer assigned to the course containing the assignment.
2. Choose the whole-class extension option in the updated lecturer menu.
3. Enter the assignment ID. The application checks that the assignment belongs to the lecturer.
4. Enter the revised deadline in YYYY-MM-DD HH:MM:SS format and a reason. The revised deadline must be later than the original assignment deadline.
5. The application applies the deadline to currently enrolled students who do not already have individual extensions and reports the number of records created or updated.
6. Existing individual extensions are preserved as exceptions. A later class extension must not shorten an existing class extension.

Student: view the class extension

Choose Courses/assignments. The assignment row displays the effective deadline and one of the extension labels: Class extension, Individual extension or No extension. Use the effective date shown by the system when planning a submission or resubmission.

5.8 Release notes — v1.1

Release type: perfective maintenance / enhancement. The v1.1 tag must be created only after PR review, CI pass and regression acceptance.

- Lecturer can grant a revised deadline to the whole class associated with an assignment.
- Class extension is applied to currently enrolled students in one transaction.
- Student assignment list displays the effective deadline and whether it is class-wide or individual.
- extension_scope distinguishes individual and class grants.
- Existing individual extensions are preserved as exceptions during a class grant.
- Date/time parsing and late-day calculations are strict and timezone/DST-independent.
- Resubmission after the effective deadline is rejected; first attempts remain subject to the late policy.

Regression testing must include all Catch2 cases and the regression matrix in Appendix C. Do not tag v1.1 if a Critical/High defect remains open or the supervisor has not accepted the overlapping-extension policy.

6. Cross-Phase Requirements Traceability

The following matrix connects the Phase 2 requirements to the workflow, implementation elements and verification evidence. The full machine-readable traceability matrix is included in the repository at docs/TRACEABILITY_MATRIX.md.

Requirement	Workflow	Design / data	Source	Verification
FR-01	Login	User hierarchy; user_account.password_hash	src/main.cpp	T02; SAT-01/02
FR-02	Role permissions	User, Administrator, Lecturer, HoD, Student	src/main.cpp	SAT-03/04/05; REG-11
FR-03	Manage accounts	user_account	src/main.cpp	SAT-06
FR-04	Departments and courses	department, course	src/main.cpp; database/schema.sql	SAT-07
FR-05	Lecturer assignment / enrolment	lecturer_course, enrolment	src/main.cpp; database/schema.sql	SAT-07
FR-06	View enrolled assignments	enrolment, course, assignment	src/main.cpp	SAT-03; SAT-19
FR-07	Create assignment	assignment; lecturer-course relation	src/main.cpp	SAT-07
FR-08	Individual and class extensions	extension_grant; extension_scope	src/main.cpp; business_rules.cpp	SAT-08/19; REG-01–07
FR-09	Submit assignment	submission; controlled files directory	src/main.cpp	SAT-09; SAT-18
FR-10	Resubmit by effective deadline	submission.version_no; effective deadline	src/main.cpp; business_rules.cpp	SAT-10; REG-09
FR-11	Late penalty	late-penalty policy function	business_rules.cpp	Boundary unit tests; SAT-09
FR-12	Mark and feedback	mark_record	src/main.cpp	SAT-11; SAT-14
FR-13	Moderate marks	mark_record.moderation	src/main.cpp	SAT-12/13
FR-14	Release approved marks	released_by; submission.status	src/main.cpp	SAT-13/14
FR-15	Audit mark changes	audit_log	src/main.cpp	SAT-14
FR-16	View own released results	mark_record, submission, user_account	src/main.cpp	SAT-03; SAT-13
FR-17	Submission states	submission.status; moderation status	src/main.cpp	SAT-09–13
FR-18	Staff work queues	course/department-scoped queries	src/main.cpp	SAT-04/05/12
NFR-01	Performance	Indexed SQLite reads	database/schema.sql	SAT-16; target-PC measurement
NFR-02	Offline use	Local executable, SQLite and files	src/main.cpp	SAT-15; REG-13

Requirement	Workflow	Design / data	Source	Verification
NFR-03	Password protection	PBKDF2-HMAC-SHA256 with random salt	src/main.cpp	T02; database inspection
NFR-04	Access control	Role and scope checks	src/main.cpp; business_rules.cpp	SAT-03/04/05; REG-03/11
NFR-05	Integrity	CHECK/FK/UNIQUE; transactions	schema; application; business_rules.cpp	SAT-11/18; CR1 integration tests
NFR-06	Auditability	Audit event with actor/time/old/new/reason	src/main.cpp	SAT-14
NFR-07	Usability	Role guides and console workflows	docs/phase7/ USER_MANUAL.md	Observed task <5 minutes per role
NFR-08	C++17 compatibility	CMake build	CMakeLists.txt; GitHub Actions	Clean build/target smoke test
NFR-09	Recoverability	Database + submission backup procedure	INSTALLATION_GUIDE.md	SAT-17; timed restore

7. Professional, Legal and Ethical Considerations

The LMS stores personal information, account credentials, enrolment information, submitted assessment files, marks, feedback and audit events. The school must therefore control access to learner information, secure assessment records and use the system only for its stated educational purpose.

7.1 Privacy and data protection

Collect only information needed to administer the school's courses and assessments. Restrict raw database and submission-folder access to authorised staff, protect backups with encryption and access controls, avoid using real learner records in development/testing, and define retention and secure disposal periods with the school. Before live use, the institution should obtain a local legal review to identify the applicable Lesotho data-protection and education-record obligations; this technical report does not substitute for legal advice or claim that a specific statutory compliance audit has been completed.

7.2 Assessment integrity and security

Role and department scope, salted password hashes, parameterised statements, foreign-key constraints, transactions, audit trails and the HoD release gate help protect assessment integrity. Default credentials must be replaced before real learners are entered. The current console displays passwords while typed and does not yet provide a password-change/reset screen; these limitations must be resolved or mitigated before production deployment.

7.3 Accessibility and inclusion

The current interface is console-based and has not yet undergone formal accessibility testing. Provide clear instructions, consistent menu wording, readable text, keyboard-only operation and staff assistance for users who need it. Acceptance testing should include users with varied levels of digital literacy and, where possible, users who rely on assistive technologies. A graphical interface would require a separate accessibility review.

7.4 Fairness of late penalties

The late penalty applies the same stated formula to each student: 10% of the assignment maximum per late calendar day for up to three days, with zero maximum after that. Individual extensions and the CR1 class-wide extension capability can accommodate approved circumstances. The school should communicate the policy before assessment deadlines, clarify calendar-day semantics, ensure extension decisions are recorded, and provide an auditable process for correcting errors or reviewing exceptional cases.

8. Conclusion and Outstanding Actions

The Phase 5–8 materials extend the Moderated LMS from implementation into verification, quality assurance, deployment planning and maintenance. The V-Model strategy and test artefacts map the SRS to unit, integration, system and acceptance testing. Configuration-management artefacts define GitHub Flow, CI, static analysis and self-review. Deployment documentation explains the single-workstation offline architecture and the technician and role-based procedures. CR1 is classified as perfective maintenance and extends the individual-extension mechanism to a whole class while preserving specific individual exceptions and displaying effective deadlines to students.

The package includes reproducible local smoke evidence, schema parity and migration checks, CR1 end-to-end output, and a preliminary coverage measurement. It does not claim completion of the unrun Catch2/CI tests or target-workstation acceptance activities. Appendix E provides the detailed execution ledger and Appendix F the cross-phase release-readiness status.

- Commit the updated Phase 5–8 source, schema, tests, documentation and workflow to GitHub on a feature branch; review and merge by pull request.
- Run GitHub Actions and retain the workflow URL, commit SHA, test log and coverage artefact; achieve at least 70% business-logic line coverage in the Catch2/CI report.
- Run all system and acceptance tests on the Windows/school target computer and fill actual result, status, tester, date and evidence columns.
- Measure NFR-01 with 800 synthetic student records on target hardware; observe NFR-07 with representative users; complete and time NFR-09 backup/restore.
- Run clang-tidy and Lizard if required; record actual findings and measured complexity rather than estimates.
- Complete the student/reviewer names, review date, exact commit SHAs, CI links and evidence references.
- Confirm with the supervisor the class-vs-individual extension precedence and the treatment of students enrolled after a class extension.
- Create v1.0 and v1.1 tags only after the respective builds, reviews and acceptance checks are complete.

References

National University of Lesotho (2026) CS4433: Software Engineering — Final Project Individual Assignment. Department of Mathematics and Computer Science, Faculty of Science and Technology, pp. 3–5.

National University of Lesotho (2026) CS4433 Software Engineering — Phase 2: Requirements Engineering. Moderated Learning Management System requirements specification.

National University of Lesotho (2026) CS4433 Software Engineering — Phase 3: System Design. Moderated Learning Management System design document.

BakoenaT (2026) Moderated-Learning-Management-System. GitHub repository. Available at: https://github.com/BakoenaT/Moderated-Learning-Management-System (accessed 1 November 2026).

Appendix A — System and acceptance test cases and actual outcomes

The following cases are designed from the use cases and requirements. Actual results are filled where the exact scope was executed; rule-level passes are distinguished from complete system

acceptance. Remaining target-machine and unexecuted cases are explicitly marked Not Run or Partial.

ID	Scenario / procedure	Expected result	Actual result	Status / evidence
SAT-01	Login — active account	Enter valid Admin, Lecturer and Student credentials.	Correct role dashboard opens.	PASS — Valid role menus opened. CR1_END_TO_END_TEST_OUTPUT.txt; 1 Nov 2026.
SAT-02	Login — invalid password	Enter an incorrect password.	Access denied; no protected data displayed.	NOT RUN — Target-machine test and evidence pending.
SAT-03	Student data isolation	Request another student's enrolments/results.	No other student data is returned.	NOT RUN — Target-machine test and evidence pending.
SAT-04	Lecturer course scope	Lecturer attempts an unassigned course action.	Request denied before write.	NOT RUN — End-to-end application test pending.
SAT-05	HoD department scope	HoD attempts to moderate another department.	Request denied.	NOT RUN — Target-machine test pending.
SAT-06	Inactive account	Deactivate account, then attempt login.	Inactive account cannot authenticate.	NOT RUN — Target-machine test pending.
SAT-07	Course and assignment setup	Admin creates department/course, assigns lecturer, enrols students; lecturer creates assignment.	Records persist; maximum mark is positive.	PASS — Setup and assignment creation completed in CR1 console scenario; 1 Nov 2026.
SAT-08	Individual extension	Assigned lecturer grants a student-specific extension with reason.	Only selected student's effective deadline changes.	PASS — Student A received individual override after class extension; CR1 E2E; 1 Nov 2026.
SAT-09	Submission and late policy	Exercise first-attempt boundary and 0–4 late calendar days.	Penalty: 0%, 10%, 20%, 30%; >3 days zero, according to adopted rule.	PARTIAL — Date/penalty logic passed; complete file-submission UI not run. SMOKE_TEST_OUTPUT.txt.
SAT-10	Resubmission boundary	Resubmit at effective deadline and one second after.	At deadline accepted; after deadline rejected.	PARTIAL — Business-rule boundary passed; full UI workflow not run. SMOKE_TEST_OUTPUT.txt.
SAT-11	Mark and feedback	Lecturer enters valid and invalid marks with feedback.	Out-of-range marks rejected; valid mark/feedback saved.	NOT RUN — Full application workflow pending.
SAT-12	Moderation return	HoD returns mark with a reason.	Mark cannot be released until corrected and approved.	NOT RUN — Full application workflow pending.
SAT-13	Release and result visibility	Student checks before and after HoD release.	Hidden before release; visible only to correct student afterwards.	NOT RUN — Full application workflow pending.
SAT-14	Mark audit	Change a mark with a reason.	Audit records actor, time, old/new mark and reason.	NOT RUN — Full application workflow pending.
SAT-15	Offline operation	Disconnect internet; complete core user workflows.	Core workflows operate without internet.	NOT RUN — Target workstation required.
SAT-16	800-student performance	Seed representative records and time common reads.	95% of common reads complete within 2 seconds.	NOT RUN — School hardware benchmark pending.
SAT-17	Backup and recovery	Back up database/submissions and restore to clean location.	Restore completes within 15 minutes; records/files reconcile.	NOT RUN — Timed target-machine evidence pending.
SAT-18	Exception handling	Trigger invalid path or database constraint failure.	Error handled; no partial transaction remains.	NOT RUN — Full application exception scenario pending.
SAT-19	CR1 class extension display	Grant whole-class extension; restart; inspect two students.	Eligible students see revised deadline and source label.	PASS — Student B saw CLASS deadline; Student A saw INDIVIDUAL override after restart. CR1 E2E; 1 Nov 2026.
SAT-20	CR1 authorisation	Unassigned lecturer attempts class extension.	Rejected; no extension rows changed.	PASS — Business-rule SQLite integration scope only. SMOKE_TEST_OUTPUT.txt; 1 Nov 2026.

Appendix B — Defect log and static-analysis results

Initial defects were identified during code review and CR1 design review. The fixes have passed the stated local rule-level or console retests; final CI and target-platform retesting remain pending.

ID	Source	Defect	Severity	Fix	Retest result	Status
D-001	Code review before Phase 5	Resubmission path did not explicitly reject attempts after effective deadline.	High	Added submissionTimeAllowed check; first attempt keeps late policy.	Manual boundary assertions passed: deadline allowed; +1 second rejected. Catch2/CI pending.	Fixed; local retest pass; CI/target pending

ID	Source	Defect	Severity	Fix	Retest result	Status
D-002	Code review before Phase 5	Invalid date input could be treated as zero late days.	Medium	Added strict Gregorian ISO date/time validation.	Manual parser assertions rejected invalid leap day, month and hour. Catch2/CI pending.	Fixed; local retest pass; CI/target pending
D-003	CR1 design review	No class-wide extension option or visible extension source.	High	Added bulk class grant, extension_scope, individual exception precedence and student list labels.	CR1 SQLite tests and console E2E passed for class/individual scope, restart display and rollback. Catch2/CI pending.	Implemented; local retest pass; CI/target pending

Static analysis record

Tool	Version / scope	Finding / result	Action / evidence required	Status
GCC build	GCC 14.2.0; all C++ build targets	No warnings in captured configured build output.	No action required; retain build log.	PASS
Clang Static Analyzer	Clang 17; main.cpp and business_rules.cpp	No diagnostics emitted.	No action required; retain analyser output.	PASS
clang-tidy	Not installed locally; business rules source	Not executed; no result claimed.	Run configured clang-tidy CI job and retain output.	PENDING
Catch2 / CTest	Catch2 v3.7.1 FetchContent; test source present	Clone failed because environment could not resolve github.com; test suite not run.	Run on network-enabled environment or GitHub Actions.	BLOCKED / PENDING
GitHub Actions	Workflow file prepared; build/test/coverage/static-analysis jobs	No Actions run evidence yet; workflow upload/commit pending.	Push workflow and source, run pipeline, retain URL and artefacts.	PENDING
Lizard metrics	Not installed locally; src and tests	Tool-generated complexity score unavailable.	Run Lizard and replace manual estimates if required.	PENDING

Appendix C — CR1 Regression Tests

Regression cases have been updated with actual local outcomes. PASS means the stated test scope was run; PARTIAL means only the underlying business rule was exercised; NOT RUN items remain open for the full application/target environment.

ID	Regression area / procedure	Expected result	Actual result	Status / evidence
REG-01	Class extension	Assigned lecturer grants extension to course with multiple enrolled students.	Eligible enrolled students receive revised deadline.	PASS — Two current students received class deadline. CR1 E2E + smoke; 1 Nov 2026.
REG-02	Individual exception	One student already has an INDIVIDUAL extension before class grant.	Individual deadline remains; others receive class deadline.	PASS — Existing individual row preserved. SMOKE_TEST_OUTPUT.txt; 1 Nov 2026.
REG-03	Assignment ownership	Unassigned lecturer attempts a class extension.	Rejected; no rows change.	PASS — Service/integration scope. SMOKE_TEST_OUTPUT.txt; 1 Nov 2026.
REG-04	Empty class	Assigned lecturer grants extension when no students are enrolled.	Validation error; no write.	PASS — Service/integration scope. SMOKE_TEST_OUTPUT.txt; 1 Nov 2026.
REG-05	Student list display	Student opens assignment list after class extension.	Effective deadline and Class extension label visible.	PASS — Display verified after app restart. CR1 E2E; 1 Nov 2026.
REG-06	Later class extension	Grant a later class deadline; attempt to shorten it.	CLASS rows update atomically; shortening rejected; INDIVIDUAL rows unchanged.	PASS — Business-rule integration. SMOKE_TEST_OUTPUT.txt; 1 Nov 2026.
REG-07	Individual after class grant	Lecturer adds individual extension after class extension.	Selected student uses individual deadline and label.	PASS — Student A saw individual deadline after restart. CR1 E2E; 1 Nov 2026.
REG-08	Late submission rule	Test late days 0–4 against penalty.	Penalty ceiling follows late policy; >3 days is zero.	PARTIAL — Logic boundary passed; DB-backed submission UI not run. SMOKE_TEST_OUTPUT.txt.
REG-09	Resubmission rule	Try resubmission at deadline and after deadline.	At-deadline allowed; after-deadline rejected.	PARTIAL — Predicate tested; console upload workflow not run. SMOKE_TEST_OUTPUT.txt.
REG-10	Marks and audit	Lecturer marks; HoD moderates/releases; audit inspected.	Visibility and audit remain correct.	NOT RUN — Full application regression pending.
REG-11	Cross-role security	Attempt cross-role and cross-department actions.	Unauthorised operations denied.	NOT RUN — Broad role matrix pending; one unassigned lecturer rule tested separately.

ID	Regression area / procedure	Expected result	Actual result	Status / evidence
REG-12	Persistence after restart	Close and reopen app after extension grant.	Class and individual scopes/deadlines persist.	PASS — Both students saw correct source/deadline after restart. CR1 E2E; 1 Nov 2026.
REG-13	Offline workflow	Disable internet and repeat core workflows.	Core local operations remain available.	NOT RUN — Target workstation test pending.
REG-14	Transaction rollback	Force a row write to fail during class extension.	Entire transaction rolls back; no partial class grant.	PASS — Trigger-induced failure rolled back all CLASS rows. SMOKE_TEST_OUTPUT.txt; 1 Nov 2026.

Appendix D — Repository Artefact Inventory and Uploaded Evidence

Path	Purpose
.github/workflows/ci.yml	Build, test, 70% business-logic coverage threshold and clang-tidy workflow.
.clang-tidy	Static-analysis configuration.
CMakeLists.txt	C++17 build and CTest configuration.
src/main.cpp	Console application, authentication and role workflows.
src/business_rules.hpp / .cpp	Strict date handling, late penalty, resubmission eligibility and CR1 class extension rules.
database/schema.sql	SQLite schema for clean database creation.
tests/test_business_rules.cpp	Catch2 unit and integration tests.
tests/manual_business_rules_smoke.cpp	Reproducible local assertion harness used for preliminary coverage evidence.
tests/seed_800_students.sql	Synthetic test data for target-machine performance testing.
docs/TRACEABILITY_MATRIX.md	Cross-phase requirement-to-source-to-test traceability.
docs/phase5/	Test strategy, acceptance tests, defect log, local test/coverage evidence.
docs/phase6/	Branching/CI strategy, metrics, static analysis and code-review record.
docs/phase7/	Deployment diagram, technician guide, role manual and v1.0 release notes.
docs/phase8/	CR1 change analysis, updated UML sources, regression tests and v1.1 notes.

Appendix E — Executed Test Results and Coverage Record

This appendix records the tests that were actually executed in the development container on 1 November 2026. It distinguishes unit/business-rule evidence, SQLite integration checks, and console end-to-end evidence from the Catch2/CI and target-workstation activities that remain pending. A PASS applies only to the stated test scope and environment.

E.1 Execution environment

Item	Observed environment
Operating system / architecture	Linux x86-64 development container (not the student Windows target PC)
CMake / compiler	CMake 3.31.6; GCC 14.2.0; C++17
Database	SQLite 3.46.1
Static analyser	Clang 17.0.0 Static Analyzer
Automated test framework	Catch2 v3.7.1 declared in CMake; not fetched or executed because DNS resolution for github.com failed
Date	1 November 2026

E.2 Executed unit and integration test results

ID	Area	Executed input / procedure	Actual result	Status
UT-01	Calendar-day boundaries	Same day, early, 1/2/3/4 days late and leap-day interval	Expected values returned for all cases.	PASS
UT-02	Strict date/time validation	Invalid leap day, month 13 and hour 25	All invalid values raised std::invalid_argument.	PASS
UT-03	Late-penalty boundary	100-mark maximum at 0/1/2/3/4 late days; negative inputs	100/90/80/70/0 ceilings; invalid negatives rejected.	PASS
UT-04	Resubmission deadline	Before, exactly at and one second after effective deadline; first late attempt	Resubmissions before/on deadline allowed; after deadline rejected; first attempt remains eligible for late handling.	PASS
IT-01	Class extension roster	Three enrolled students; one pre-existing individual extension; unrelated assignment	Two eligible students received CLASS scope; individual row and unrelated assignment unchanged.	PASS
IT-02	Ownership and input validation	Null DB, invalid IDs, wrong lecturer, malformed deadline, empty reason, early deadline, empty class	Every invalid or unauthorised request rejected.	PASS
IT-03	Extension labels	CLASS, INDIVIDUAL and empty scope	Correct class/individual/no-extension labels returned.	PASS
IT-04	Atomic rollback	SQLite trigger aborts one row write during class update	Entire class grant rolled back; no partial CLASS rows remained.	PASS
IT-05	Later class extension	Grant later class deadline after first class grant; reject shorter deadline	CLASS rows updated to later date; shortening rejected; INDIVIDUAL exception unaffected.	PASS

E.3 Console end-to-end result — CR1

A scripted console run created a department, a Lecturer, two Students, a course and enrolments. The Lecturer created an assignment and granted a class-wide extension, then granted a later individual extension to Student A. The application exited. In a new process, Student A and Student B logged in and viewed their assignment lists. SQLite was inspected after restart.

Student	Stored extension scope	Effective deadline shown after restart	Observed label	Result
Student A	INDIVIDUAL	2026-11-14 12:00:00	Individual extension	PASS
Student B	CLASS	2026-11-12 12:00:00	Class extension	PASS

The class extension was applied to both enrolled students. The individual grant was then retained as a student-specific exception. Both effective deadlines and labels were still available after closing and reopening the application. The terminal transcript is retained in docs/phase8/CR1_END_TO_END_TEST_OUTPUT.txt.

E.4 Database and migration results

Test	Actual result	Status
Application initialization	The production executable initialized ten expected LMS tables and extension_scope.	PASS
Schema parity	Runtime SQL and database/schema.sql matched for all ten table column and foreign-key definitions.	PASS
Foreign-key check	PRAGMA foreign_key_check returned no violations for the separately created schemas.	PASS
Legacy migration	Existing extension_grant table without extension_scope was upgraded; old row preserved with default scope INDIVIDUAL.	PASS

E.5 Coverage result

The manual assertion harness was compiled with GCC coverage instrumentation and executed against src/business_rules.cpp. gcov reported 84.18% line execution of 158 executable lines and 78.91% of 294 branches executed; 49.66% of branches were taken at least once. This is preliminary smoke-harness evidence only. The Catch2/CTest suite did not run, so this is not the final CI coverage result and must not be described as a Catch2 coverage report.

E.6 Blocked and unexecuted test activities

Activity	Status	Reason / next evidence required
Catch2 v3 / CTest	BLOCKED / PENDING	CMake FetchContent failed to clone Catch2 because this container could not resolve github.com. Run on network-enabled machine or GitHub Actions.
GitHub Actions build/test/coverage	PENDING	Upload the workflow and phase source/tests, commit to a branch, run CI and retain URL, SHA, test log and coverage artifact.
Remaining system/acceptance cases	NOT RUN	Complete SAT-02 to SAT-06 and SAT-11 to SAT-18 on the target workstation; record tester, date, actual result and evidence.
800-student performance	NOT RUN	Seed representative synthetic data and measure 95th percentile on the school workstation against the 2-second target.
Offline usability and backup/restore	NOT RUN	Disconnect internet, observe at least one user per role and time restore against the 15-minute target.

Appendix F — Phase 6–8 Verification and Release Readiness

F.1 Phase 6 — Configuration management and QA results

Requirement	Prepared artefact	Observed result / status
Git repository and branching	Repository link and GitHub Flow strategy	Repository exists. The exact final commit SHA and pull request for Phase 5–8 updates must be recorded after upload.
CI pipeline	`.github/workflows/ci.yml`	Workflow prepared in the package; no successful GitHub Actions run is claimed.
Static analysis	GCC warning build and Clang 17 Static Analyzer	PASS: no GCC warnings in captured build and no Clang Static Analyzer diagnostics. clang-tidy remains pending.
Code metrics	Physical line counts and complexity table	643 physical lines across five measured C++ source/test files. Complexity values are manual estimates, not Lizard output.
Self-code review	Checklist and action record	Local review findings/actions recorded; student reviewer initials and final SHA remain to be completed.
Release tagging	v1.0 and v1.1 tag procedure	Tags are proposed only; do not create until build, CI, acceptance and review gates pass.

F.2 Phase 7 — Deployment verification

Deliverable / check	Status	Evidence / qualification
UML deployment diagram	PREPARED	deployment_diagram.puml, .dot, .svg and .png included; diagram visually checked.
Technician installation guide	PREPARED	Windows/MSVC + vcpkg and Ubuntu/Debian procedures documented.
Four role user manuals	PREPARED	Administrator, Lecturer, HoD and Student guidance included.
v1.0 release notes	PREPARED	Release notes supplied; Git tag and production release not created.
Local Linux build and database initialization	PASS	CMake/GCC build and first-run DB initialization succeeded in the development container.
Windows/school workstation installation	NOT RUN	Windows MSVC compiler/dependency setup and clean target build remain outstanding.
School acceptance and backup/restore	NOT RUN	Require representative users and timed restore evidence on target hardware.

F.3 Phase 8 — CR1 regression execution results

CR1 is classified as perfective maintenance / functional enhancement. The updated implementation applies a class extension to currently enrolled students in one SQLite transaction, preserves individual exceptions, allows a later class deadline to update CLASS rows without shortening an existing grant, and displays the effective deadline and extension source to students.

Regression ID	Actual result	Status
REG-01	Class extension applied to eligible class members in console workflow and SQLite test fixture.	PASS
REG-02	Existing individual extension was preserved during class grant.	PASS
REG-03	Lecturer not assigned to the assignment was rejected at the business-rule boundary.	PASS — service scope
REG-04	Empty class was rejected with no extension write.	PASS — service scope
REG-05	Student assignment list displayed the class deadline and label after restart.	PASS
REG-06	Later class grant updated CLASS rows; shorter grant rejected; individual exception unchanged.	PASS
REG-07	Individual extension after class grant became the selected student’s effective deadline.	PASS
REG-08	Late-day/penalty rule boundaries passed; database-backed file submission sequence not run.	PARTIAL

Regression ID	Actual result	Status
REG-09	Resubmission rule boundary passed at/before/after deadline; full upload menu path not run.	PARTIAL
REG-10	Marking, moderation, release and audit regression not executed in CR1 run.	NOT RUN
REG-11	Broad cross-role/cross-department regression not executed; only lecturer ownership case exercised.	NOT RUN / PARTIAL
REG-12	Extension records and display survived application restart.	PASS
REG-13	Offline workflow not explicitly run on target machine.	NOT RUN
REG-14	Forced database trigger failure rolled back all class extension rows.	PASS

F.4 Release decision

The CR1 feature is implemented in the Phase 5–8 package and its core rule, transaction, student-display and restart scenarios passed local tests. However, the release is not yet final: Catch2/CTest and GitHub Actions have not completed; full existing-feature regression, Windows target testing, performance, usability and backup/restore remain outstanding. The v1.1 tag must wait for these gates and supervisor confirmation of class-vs-individual precedence and the treatment of students enrolled after a class grant.